

LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK
FUNGSI CRUD DENGAN DATABASE MySQL 2



OLEH :

TEGUH ESA MAULANNA

2411532005

DOSEN PENGAMPU:

NURFIAH, S.ST, M.Kom.,

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG 2025

A. Pendahuluan

MySQL merupakan sebuah manajemen basis data yang menggunakan perintah dasar SQL (Structured Query Language). SQL merupakan suatu sintak perintah tertentu yang digunakan untuk mengelola suatu database. SQL merupakan bahasa standar yang digunakan dalam manajemen basis data relasional atau disebut sebagai Relational Database Management System (RDBMS).

MySQL merupakan sebuah driver atau library yang digunakan untuk dengan sistem manajemen MySQL seperti Java, python, PHP, C++, dan .NET. Fungsinya adalah sebagai perantara untuk komunikasi antara aplikasi dan database, sehingga operasi seperti membaca, menulis, dan memodifikasi dapat dilakukan dengan lebih praktis dan efisien.

DAO atau Data Access Object adalah sebuah cara untuk mengatur kode program agar dapat menangani komunikasi antara aplikasi dan basis data. DAO berisikan objek yang menyediakan abstract interface untuk beberapa method yang berhubungan dengan database seperti mengambil data atau membaca data (read), menyimpan data(create), menghapus data (delete), mengubah data(update).

CRUD merupakan singkatan dari Create, Read, Update, dan Delete merupakan sebuah operasi dasar dalam pengelolaan data dalam sebuah database terkhusus pada database yang bersifat rasional. CRUD merupakan pondasi dasar dalam pengembangan sistem atau aplikasi yang berhubungan dengan pengolahan data.

B. Tujuan

1. Memahami konsep dasar MySQL sebagai sistem manajemen basis data.
2. Mempelajari fungsi driver atau library MySQL Connector sebagai penghubung antara aplikasi dan database.
3. Mengimplementasikan operasi dasar CRUD pada pemrograman berbasis objek.

C. Langkah Kerja Praktikum

- a) Menggunakan fungsi CRUD pada GUI.
 1. Membuat method reset.

```
public void reset() {  
    txtName.setText("");  
    txtUsername.setText("");  
    txtPassword.setText("");  
}
```

Method ini digunakan untuk menghapus semua inputan yaitu name, username, dan password, lalu mengembalikannya dalam kondisi kosong.

2. Membuat instance pada GUI.

```
UserRepo usr = new UserRepo();
List<User> ls;
public String id;
```

Hal ini bertujuan untuk membuat sebuah objek dari kelas UserRepo. Mendeklarasikan sebuah list yang berisikan objek dari kelas user. Mendeklarasikan variable id dengan tipe data string yang dapat diakses secara public.

b) Membuat objek user.

1. Memberikan respon pada tombol save.

```
JButton btnSave = new JButton("SAVE");
btnSave.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        User user = new User();
        user.setNama(txtName.getText());
        user.setUsername(txtUsername.getText());
        user.setPassword(txtPassword.getText());
        usr.save(user);
        reset();
        loadTable();
    }
});
```

Hal ini berguna untuk membuat sebuah objek baru dari user. Mengambil inputan dari user berupa nama, username, dan password lalu menyimpannya dalam variable user. Setelah data tersimpan panggil method reset untuk mengosongkan inputan, dan menyimpannya kedalam tabel.

c) Membaca objek user.

1. Membaca objek user

```
public void loadTable() {
    ls = usr.show();
    TableUser tu = new TableUser(ls);
    tableUsers.setModel(tu);
    tableUsers.getTableHeader().setVisible(true);
}
```

Sebuah method untuk menampilkan data user kedalam tabel. Panggil variable ls yang berisikan list user dan menampilkan objek usr dari userRepo. Buat sebuah tabel dengan nama tu dan mengisinya dengan daftar list dari ls.

2. Menampilkan tabel kedalam userFrame

```
UserFrame frame = new UserFrame();
frame.setVisible(true);
frame.loadTable();
```

Ketika userFrame dibuka, isikan tabel dengan data dari data user yang ada di database.

d) Memperbarui objek user.

1. Memberikan respon pada tabel.

```
tableUsers = new JTable();
tableUsers.addMouseListener(new MouseAdapter() {
    @Override
    public void mouseClicked(MouseEvent e) {
        id = tableUsers.getValueAt(tableUsers.getSelectedRow(),0).toString();
        txtName.setText(tableUsers.getValueAt(tableUsers.getSelectedRow(),1).toString());
        txtUsername.setText(tableUsers.getValueAt(tableUsers.getSelectedRow(),2).toString());
        txtPassword.setText(tableUsers.getValueAt(tableUsers.getSelectedRow(),3).toString());
    }
});
scrollPane.setViewportView(tableUsers);
```

Membuat sebuah tabel baru dan respon berupa mouse click. Setiap baris tabel diberikan sebuah id, sehingga ketika pengguna mengklik sebuah baris otomatis menampilkan informasi kedalam inputan pengguna.

2. Memberikan respon pada tombol update.

```
JButton btnUpdate = new JButton("UPDATE");
btnUpdate.setFont(new Font("Times New Roman", Font.BOLD, 10));
btnUpdate.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        User user = new User();
        user.setNama(txtName.getText());
        user.setUsername(txtUsername.getText());
        user.setPassword(txtPassword.getText());
        user.setId(id);
        usr.update(user);
        reset();
        loadTable();
    }
});
```

Membuat sebuah objek baru dari user lalu mengambil input dari pengguna berupa nama, username, password. Objek baru tersebut diberikan sebuah id agar sistem tahu mana yang harus diperbarui. Setelah selesai memperbarui kosongkan kolom inputan pengguna dan simpan data kedalam tabel.

e) Menghapus objek user.

1. Memberikan respon pada tombol delete.

```
JButton btnDelete = new JButton("DELETE");
btnDelete.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if(id != null) {
            usr.delete(id);
            reset();
            loadTable();
        } else {
            JOptionPane.showMessageDialog(null, "Silahkan pilih data yang akan di hapus");
        }
    }
});
```

Program akan melakukan pengecekan terlebih dahulu untuk memastikan sudah ada objek yang dipilih menggunakan if – else statement. Jika id tidak kosong panggil method delete berdasarkan id yang dipilih, lalu kosongkan kotak inputan dan perbarui tabel. Jika id tidak ada atau kosong program menampilkan pesan berupa “silahkan pilih data yang akan di hapus”.

f) Output program

Users

Name

Username

Password

SAVE

UPDATE

DELETE

CANCEL

ID	Name	Username	Password
----	------	----------	----------

Users

Name

Teguh

Username

teguh123

Password

12345

SAVE

UPDATE

DELETE

CANCEL

ID	Name	Username	Password
----	------	----------	----------


Users
—
□
×

Name

Username

Password

SAVE

UPDATE

DELETE

CANCEL

ID	Name	Username	Password
2	Teguh	teguh123	12345

g) Latihan atau Tugas

1. Buat tabel customer dan service baru dalam database.

Customer

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	id 	varchar(10)	utf8mb4_general_ci		No	None			 Change  Drop  More
☐ 2	nama	varchar(100)	utf8mb4_general_ci		No	None			 Change  Drop  More
☐ 3	alamat	varchar(200)	utf8mb4_general_ci		No	None			 Change  Drop  More
☐ 4	nomorhp	varchar(20)	utf8mb4_general_ci		No	None			 Change  Drop  More

Tabel terdiri dari id(varchar), nama(varchar), Alamat(varchar), dan nomorHp(varchar).

Service

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 id	varchar(10)	utf8mb4_general_ci		No	None			Change Drop More
☐	2 jenis	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 harga	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
☐	4 status	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More

Tabel terdiri dari id(varchar), jenis(varchar), harga(varchar), dan status(varchar).

2. Buat tampilan CRUD untuk customer dan service.
3. Buat tabel model untuk customer dan service.

Customer

```
package table;

import java.util.List;

public class TableCustomer extends AbstractTableModel {
    List<Customer> ls;
    private String[] columnNames = {"ID", "Nama", "Alamat", "Nomor HP"};

    public TableCustomer(List<Customer> ls) {
        this.ls = ls;
    }

    @Override
    public int getRowCount() {
        return ls.size();
    }

    @Override
    public int getColumnCount() {
        return 4;
    }

    @Override
    public String getColumnName(int column) {
        return columnNames[column];
    }

    @Override
    public Object getValueAt(int rowIndex, int columnIndex) {
        switch (columnIndex) {
            case 0:
                return ls.get(rowIndex).getId();
            case 1:
                return ls.get(rowIndex).getNama();
            case 2:
                return ls.get(rowIndex).getAlamat();
            case 3:
                return ls.get(rowIndex).getNomorhp();
            default:
                return null;
        }
    }
}
```

Kelas ini berfungsi sebagai penghubung dari data objek customer dengan JTable. Data akan disimpan dalam list<Customer>. Sehingga data yang disimpan dapat ditampilkan dalam bentuk tabel pada halaman GUI.

Service

```

1 package table;
2
3 import java.util.List;
4
5
6 public class TableService extends AbstractTableModel {
7     List<Service> ls;
8     private String[] columnNames = {"ID", "Jenis", "Harga", "Status"};
9
10    public TableService(List<Service> ls) {
11        this.ls = ls;
12    }
13
14    @Override
15    public int getRowCount() {
16        return ls.size();
17    }
18
19    @Override
20    public int getColumnCount() {
21        return 4;
22    }
23
24    @Override
25    public String getColumnName(int column) {
26        return columnNames[column];
27    }
28
29    @Override
30    public Object getValueAt(int rowIndex, int columnIndex) {
31        switch (columnIndex) {
32            case 0:
33                return ls.get(rowIndex).getId();
34            case 1:
35                return ls.get(rowIndex).getJenis();
36            case 2:
37                return ls.get(rowIndex).getHarga();
38            case 3:
39                return ls.get(rowIndex).getStatus();
40            default:
41                return null;
42        }
43    }
44 }

```

Kelas ini berfungsi sebagai penghubung dari data objek service dengan JTable. Data akan disimpan dalam list<Service>. Sehingga data yang disimpan dapat ditampilkan dalam bentuk tabel pada halaman GUI.

4. Membuat fungsi DAO untuk customer dan service.

Customer

```

1 package DAO;
2
3 import java.util.List;
4
5
6 public interface CustomerDAO {
7     void save(Customer customer);
8     List<Customer> show();
9     public void delete(String id);
10    public void update(Customer customer);
11 }

```

Terdiri dari beberapa operasi CRUD yaitu save untuk menyimpan segala proses operasi yang dilakukan dan menyimpannya dalam objek customer, show untuk mengambil dan menampilkan semua data dari customer dalam bentuk list, delete untuk menghapus atribut customer berdasarkan id, dan update untuk mengubah data customer dengan data yang baru.

Service


```

1 package DAO;
2
3 import java.util.List;
4
5
6 public interface ServiceDAO {
7     void save(Service service);
8     List<Service> show();
9     public void delete(String id);
10    public void update(Service service);
11 }

```

Terdiri dari beberapa operasi CRUD yaitu save untuk menyimpan segala proses operasi yang dilakukan dan menyimpannya dalam objek service, show untuk mengambil dan menampilkan semua data dari service dalam bentuk list, delete untuk menghapus atribut customer berdasarkan id, dan update untuk mengubah data service dengan data yang baru.

5. Menggunakan fungsi DAO

Customer

```

1 package DAO;
2
3 import java.sql.Connection;
4
5
6 public class CustomerRepo implements CustomerDAO {
7
8     private Connection connection;
9     final String insert = "INSERT into customer (id, nama, alamat, nomorhp) VALUES (?, ?, ?, ?)";
10    final String select = "SELECT * FROM customer";
11    final String delete = "DELETE FROM customer WHERE id=?";
12    final String update = "UPDATE customer SET nama=?, alamat=?, nomorhp=? WHERE id=?";
13
14    public CustomerRepo() {
15        connection = Database.koneksi();
16    }
17
18    @Override
19    public void save(Customer customer) {
20        PreparedStatement st = null;
21        try {
22            st = connection.prepareStatement(insert);
23            st.setString(1, customer.getId());
24            st.setString(2, customer.getNama());
25            st.setString(3, customer.getAlamat());
26            st.setString(4, customer.getNomorhp());
27            st.executeUpdate();
28        } catch (SQLException e) {
29            e.printStackTrace();
30        } finally {
31            try { if (st != null) st.close(); } catch (SQLException e) { e.printStackTrace(); }
32        }
33    }
34
35    @Override
36    public List<Customer> show() {
37        List<Customer> ls = new ArrayList<>();
38        try {
39            Statement st = connection.createStatement();
40            ResultSet rs = st.executeQuery(select);
41            while(rs.next()) {
42                Customer c = new Customer();
43                c.setId(rs.getString("id"));
44            }
45        }
46    }
47
48 }

```

```

54         c.setNama(rs.getString("nama"));
55         c.setAlamat(rs.getString("alamat"));
56         c.setNomorhp(rs.getString("nomorhp"));
57         ls.add(c);
58     }
59 } catch (SQLException e) {
60     Logger.getLogger(CustomerDAO.class.getName()).log(Level.SEVERE, null, e);
61 }
62 return ls;
63 }
64
65 @Override
66 public void delete(String id) {
67     PreparedStatement st = null;
68     try {
69         st = connection.prepareStatement(delete);
70         st.setString(1, id);
71         st.executeUpdate();
72     } catch (SQLException e) {
73         e.printStackTrace();
74     } finally {
75         try { if (st != null) st.close(); } catch (SQLException e) { e.printStackTrace(); }
76     }
77 }
78
79 @Override
80 public void update(Customer customer) {
81     PreparedStatement st = null;
82     try {
83         st = connection.prepareStatement(update);
84         st.setString(1, customer.getNama());
85         st.setString(2, customer.getAlamat());
86         st.setString(3, customer.getNomorhp());
87         st.setString(4, customer.getId());
88         st.executeUpdate();
89     } catch (SQLException e) {
90         e.printStackTrace();
91     } finally {
92         try { if (st != null) st.close(); } catch (SQLException e) { e.printStackTrace(); }
93     }
94 }
95 }

```

Kelas ini merupakan sebuah repository untuk mengelola tabel customer yang ada dalam database. Repo ini menyediakan 4 fungsi operasi yaitu save untuk menambah data baru, show untuk menampilkan semua data dalam bentuk list, delete untuk menghapus data berdasarkan id, dan update untuk mengubah data berdasarkan id.

Service

```

1 package DAO;
2
3 import java.sql.Connection;
4
15 public class ServiceRepo implements ServiceDAO {
16
17     private Connection connection;
18     final String insert = "INSERT into service (id, jenis, harga, status) VALUES (?, ?, ?, ?);";
19     final String select = "SELECT * FROM service;";
20     final String delete = "DELETE FROM service WHERE id=?;";
21     final String update = "UPDATE service SET jenis=?, harga=?, status=? WHERE id=?;";
22
23     public ServiceRepo() {
24         connection = Database.koneksi();
25     }
26
27     @Override
28     public void save(Service service) {
29         PreparedStatement st = null;
30         try {
31             st = connection.prepareStatement(insert);
32             st.setString(1, service.getId());
33             st.setString(2, service.getJenis());
34             st.setString(3, service.getHarga());
35             st.setString(4, service.getStatus());
36             st.executeUpdate();
37         } catch (SQLException e) {
38             e.printStackTrace();
39         } finally {
40             try { if (st != null) st.close(); } catch (SQLException e) { e.printStackTrace(); }
41         }
42     }
43
44     @Override
45     public List<Service> show() {
46         List<Service> ls = new ArrayList<>();
47         try {
48             Statement st = connection.createStatement();
49             ResultSet rs = st.executeQuery(select);
50             while(rs.next()) {
51                 Service s = new Service();
52                 s.setId(rs.getString("id"));
53

```

```

54         s.setJenis(rs.getString("jenis"));
55         s.setHarga(rs.getString("harga"));
56         s.setStatus(rs.getString("status"));
57         ls.add(s);
58     }
59     } catch (SQLException e) {
60         Logger.getLogger(ServiceDAO.class.getName()).log(Level.SEVERE, null, e);
61     }
62     return ls;
63 }
64
65 @Override
66 public void delete(String id) {
67     PreparedStatement st = null;
68     try {
69         st = connection.prepareStatement(delete);
70         st.setString(1, id);
71         st.executeUpdate();
72     } catch (SQLException e) {
73         e.printStackTrace();
74     } finally {
75         try { if (st != null) st.close(); } catch (SQLException e) { e.printStackTrace(); }
76     }
77 }
78
79 @Override
80 public void update(Service service) {
81     PreparedStatement st = null;
82     try {
83         st = connection.prepareStatement(update);
84         st.setString(1, service.getJenis());
85         st.setString(2, service.getHarga());
86         st.setString(3, service.getStatus());
87         st.setString(4, service.getId());
88         st.executeUpdate();
89     } catch (SQLException e) {
90         e.printStackTrace();
91     } finally {
92         try { if (st != null) st.close(); } catch (SQLException e) { e.printStackTrace(); }
93     }
94 }
95 }

```

Kelas ini merupakan sebuah repository untuk mengelola tabel service yang ada dalam database. Repo ini menyediakan 4 fungsi operasi yaitu save untuk menambah data baru, show untuk menampilkan semua data dalam bentuk list, delete untuk menghapus data berdasarkan id, dan update untuk mengubah data berdasarkan id.

D. Kesimpulan

Dari praktikum ini kita telah mempelajari bahwa MySQL merupakan sistem manajemen basis data relasional yang menggunakan SQL sebagai bahasa standar pengolahan data. MySQL connector digunakan sebagai penghubung antara aplikasi pemrograman dengan sistem basis data. Dengan menerapkan pola desain DAO, proses pengelolaan data dalam aplikasi menjadi lebih terstruktur karena logika akses data dipisahkan dari logika utama program. Selain itu, implementasi operasi CRUD sangat penting sebagai pemahaman dasar dalam mengelola data, karena menjadi pondasi bagi hampir semua aplikasi berbasis database.